

Problem 9.4.2

Sarvesh Tamgade

October 1, 2025

Question: Find the roots of the following quadratic equation graphically:

$$x(2x + 3) = x^2 + 1 \quad (1.1)$$

Solution

First, we expand and rearrange the equation to standard form:

$$x(2x + 3) = x^2 + 1 \quad (2.1)$$

$$2x^2 + 3x = x^2 + 1 \quad (2.2)$$

$$2x^2 + 3x - x^2 - 1 = 0 \quad (2.3)$$

$$x^2 + 3x - 1 = 0 \quad (2.4)$$

Input Variables:

The given quadratic can be written in the conic form:

$$\mathbf{x}^T \mathbf{V} \mathbf{x} + 2\mathbf{u}^T \mathbf{x} + f = 0 \quad (2.5)$$

where

$$\mathbf{V} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad \mathbf{u} = \begin{pmatrix} \frac{3}{2} \\ 0 \end{pmatrix}, \quad f = -1 \quad (2.6)$$

Since the roots of the quadratic correspond to intersections with the x-axis, we represent the line:

$$L : \mathbf{x} = \mathbf{h} + \kappa \mathbf{m} \quad (2.7)$$

with

$$\mathbf{h} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \mathbf{m} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad (2.8)$$

Solution

Symbol	Value
v	$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$
u	$\begin{pmatrix} \frac{3}{2} \\ 0 \end{pmatrix}$
<i>f</i>	-1
h	$\begin{pmatrix} 0 \\ 0 \end{pmatrix}$
m	$\begin{pmatrix} 1 \\ 0 \end{pmatrix}$

Table: Input Parameters

Solution

The points of intersection of a line with a conic are given by:

$$\kappa = \frac{1}{\mathbf{m}^T \mathbf{V} \mathbf{m}} \left[-\mathbf{m}^T (\mathbf{V} \mathbf{h} + \mathbf{u}) \pm \sqrt{(\mathbf{m}^T (\mathbf{V} \mathbf{h} + \mathbf{u}))^2 - g(\mathbf{h}) (\mathbf{m}^T \mathbf{V} \mathbf{m})} \right] \quad (2.9)$$

where

$$g(\mathbf{h}) = \mathbf{h}^T \mathbf{V} \mathbf{h} + 2\mathbf{u}^T \mathbf{h} + f \quad (2.10)$$

Step 1: Compute $\mathbf{m}^T \mathbf{V} \mathbf{m}$

$$\mathbf{m}^T \mathbf{V} \mathbf{m} = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = 1 \quad (2.11)$$

Step 2: Compute $\mathbf{V} \mathbf{h} + \mathbf{u}$

$$\mathbf{V} \mathbf{h} + \mathbf{u} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \begin{pmatrix} \frac{3}{2} \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{3}{2} \\ 0 \end{pmatrix} \quad (2.12)$$

Solution

Step 3: Compute $\mathbf{m}^T (\mathbf{V}\mathbf{h} + \mathbf{u})$

$$\mathbf{m}^T (\mathbf{V}\mathbf{h} + \mathbf{u}) = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} \frac{3}{2} \\ 0 \end{pmatrix} = \frac{3}{2} \quad (2.13)$$

Step 4: Compute $g(\mathbf{h})$

$$g(\mathbf{h}) = \mathbf{h}^T \mathbf{V}\mathbf{h} + 2\mathbf{u}^T \mathbf{h} + f = 0 + 0 + (-1) = -1 \quad (2.14)$$

Step 5: Substitute into formula for κ

$$\kappa = -\frac{3}{2} \pm \sqrt{\left(\frac{3}{2}\right)^2 - (-1)(1)} \quad (2.15)$$

$$= -\frac{3}{2} \pm \sqrt{\frac{9}{4} + 1} \quad (2.16)$$

$$= \frac{-3 \pm \sqrt{13}}{2} \quad (2.17)$$

Step 6: Evaluate roots

$$\kappa_1 = \frac{-3 + \sqrt{13}}{2} \quad (2.18)$$

$$\kappa_2 = \frac{-3 - \sqrt{13}}{2} \quad (2.19)$$

Step 7: Find intersection points The intersection points are obtained as:

$$\mathbf{x}_1 = \mathbf{h} + \kappa_1 \mathbf{m} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \frac{-3 + \sqrt{13}}{2} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{-3 + \sqrt{13}}{2} \\ 0 \end{pmatrix} \quad (2.20)$$

$$\mathbf{x}_2 = \mathbf{h} + \kappa_2 \mathbf{m} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \frac{-3 - \sqrt{13}}{2} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{-3 - \sqrt{13}}{2} \\ 0 \end{pmatrix} \quad (2.21)$$

Solution

Thus, the quadratic $x^2 + 3x - 1 = 0$ intersects the x-axis at:

$$x = \frac{-3 - \sqrt{13}}{2} \approx -3.303 \quad (2.22)$$

$$x = \frac{-3 + \sqrt{13}}{2} \approx 0.303 \quad (2.23)$$

The roots of the original equation $x(2x + 3) = x^2 + 1$ are:

$$\boxed{x = \frac{-3 \pm \sqrt{13}}{2}} \quad (2.24)$$

Graph

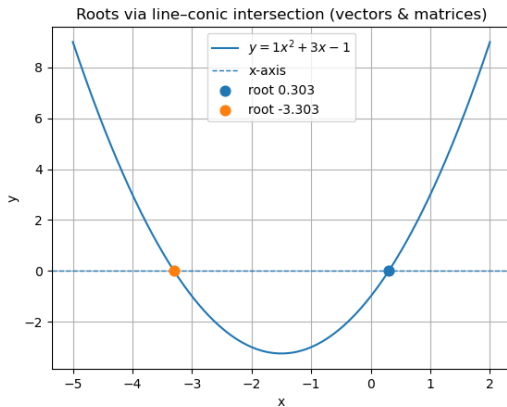


Figure: Vector Representation

```
#include <math.h>

void intersect_line_conic(
    const double V[4],
    const double u[2],
    double f,
    const double h[2],
    const double m[2],
    double kappa_out[2],
    double x1[2],
    double x2[2],
    int *status
) {
    double Vm0 = V[0]*m[0] + V[1]*m[1];
    double Vm1 = V[2]*m[0] + V[3]*m[1];
    double mTVm = m[0]*Vm0 + m[1]*Vm1;
```

```
double Vh0 = V[0]*h[0] + V[1]*h[1];
double Vh1 = V[2]*h[0] + V[3]*h[1];

double Vh_u0 = Vh0 + u[0];
double Vh_u1 = Vh1 + u[1];

double mT_Vh_u = m[0]*Vh_u0 + m[1]*Vh_u1;

double hTVh = h[0]*Vh0 + h[1]*Vh1;
double two_uTh = 2.0*(u[0]*h[0] + u[1]*h[1]);
double g = hTVh + two_uTh + f;

double disc = mT_Vh_u*mT_Vh_u - g*mTVm;
```

```
if (disc > 1e-12) {  
    double r = sqrt(disc);  
    kappa_out[0] = (-mT_Vh_u + r) / mTVm;  
    kappa_out[1] = (-mT_Vh_u - r) / mTVm;  
  
    x1[0] = h[0] + kappa_out[0]*m[0]; x1[1] = h[1] +  
        kappa_out[0]*m[1];  
    x2[0] = h[0] + kappa_out[1]*m[0]; x2[1] = h[1] +  
        kappa_out[1]*m[1];  
  
    *status = 2;  
}
```

```
    } else if (fabs(disc) <= 1e-12) {  
        kappa_out[0] = kappa_out[1] = (-mT_Vh_u) / mTVm;  
        x1[0] = x2[0] = h[0] + kappa_out[0]*m[0];  
        x1[1] = x2[1] = h[1] + kappa_out[0]*m[1];  
  
        *status = 1;  
    } else {  
        *status = 0;  
    }  
}
```

Python + C Code

```
import ctypes as ct
import numpy as np
import matplotlib.pyplot as plt

# --- load the shared library (same folder) ---
lib = ct.CDLL("./libsimple_conic.so")

lib.intersect_line_conic.argtypes = [
    ct.POINTER(ct.c_double), # V[4]
    ct.POINTER(ct.c_double), # u[2]
    ct.c_double, # f
    ct.POINTER(ct.c_double), # h[2]
    ct.POINTER(ct.c_double), # m[2]
    ct.POINTER(ct.c_double), # kappa[2] out
    ct.POINTER(ct.c_double), # x1[2] out
    ct.POINTER(ct.c_double), # x2[2] out
    ct.POINTER(ct.c_int) # status out
]
```

```
lib.intersect_line_conic.restype = None

# ---- helper: convert quadratic  $ax^2+bx+c$  into conic form ----
def quadratic_to_conic(a, b, c):
    V = np.array([a, 0.0, 0.0, 0.0], dtype=np.double) # [[a
        ,0],[0,0]]
    u = np.array([b/2.0, 0.0], dtype=np.double) # so  $2u^T x = b x$ 
    f = np.double(c)
    return V, u, f

# ---- our quadratic:  $x^2 + 3x - 1 = 0$  ----
a, b, c = 1.0, 3.0, -1.0
V, u, f = quadratic_to_conic(a, b, c)

# Intersect with x-axis ( $y=0$ ): line  $x = h + k m$ 
h = np.array([0.0, 0.0], dtype=np.double)
m = np.array([1.0, 0.0], dtype=np.double)
```

```
# Outputs (allocated for C)
kappa = np.zeros(2, dtype=np.double)
x1 = np.zeros(2, dtype=np.double)
x2 = np.zeros(2, dtype=np.double)
status = ct.c_int(0)

# ---- call the C function ----
lib.intersect_line_conic(
    V.ctypes.data_as(ct.POINTER(ct.c_double)),
    u.ctypes.data_as(ct.POINTER(ct.c_double)),
    ct.c_double(f),
    h.ctypes.data_as(ct.POINTER(ct.c_double)),
    m.ctypes.data_as(ct.POINTER(ct.c_double)),
    kappa.ctypes.data_as(ct.POINTER(ct.c_double)),
    x1.ctypes.data_as(ct.POINTER(ct.c_double)),
    x2.ctypes.data_as(ct.POINTER(ct.c_double)),
    ct.byref(status)
)
```

```
# Collect results
roots = []

if status.value >= 1:
    roots.append(float(x1[0]))
    if status.value == 2:
        roots.append(float(x2[0]))
    roots.sort()

print("status:", status.value)
print("roots:", roots) # expected [(-3-13)/2, (-3+13)/2]
                        [-3.303, 0.303]

# ---- plot parabola and roots ----
xs = np.linspace(-5, 2, 600) # adjusted range for our equation
ys = a*xs*xs + b*xs + c
```

```
plt.figure()
plt.plot(xs, ys, label=rf"$y={a:.0f}x^2{b:+.0f}x{c:+.0f}$")
plt.axhline(0, linestyle="--", linewidth=1, label="x-axis")

for r in roots:
    plt.scatter([r], [0.0], s=60, zorder=3, label=f"root {r:.3f}")

plt.xlabel("x"); plt.ylabel("y")
plt.title("Roots via lineconic intersection (NumPy + C)")
plt.grid(True)
plt.legend()
plt.savefig("parabola.png")
plt.show()
```

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

def quadratic_to_conic(a, b, c):
    V = np.array([[a, 0.0],
                  [0.0, 0.0]], dtype=float)
    u = np.array([b/2.0, 0.0], dtype=float) # so that  $2 u^T x = b$ 
    f = float(c)
    return V, u, f

def line_conic_intersection(V, u, f, h, m, eps=1e-12):
    #  $m^T V m$ 
    Vm = V @ m
    mTVm = float(m @ Vm)

    #  $Vh + u$ 
    Vh_u = V @ h + u
```

```
mT_Vh_u = float(m @ Vh_u)

#  $g(h) = h^T V h + 2 u^T h + f$ 
g = float(h @ (V @ h) + 2.0 * (u @ h) + f)

disc = mT_Vh_u**2 - g * mTVm

if disc > eps:
    r = np.sqrt(disc)
    k1 = (-mT_Vh_u + r) / mTVm
    k2 = (-mT_Vh_u - r) / mTVm

    X1 = h + k1 * m
    X2 = h + k2 * m

    return 2, np.array([k1, k2], dtype=float), np.vstack([X1,
                                                            X2])
```

Python Code

```
elif abs(disc) <= eps:
    k = (-mT_Vh_u) / mTVm
    X = h + k * m

    return 1, np.array([k, k], dtype=float), np.vstack([X, X
    ])
else:
    return 0, np.array([np.nan, np.nan]), np.array([[np.nan,
    np.nan],
    [np.nan, np.
    nan]])

# ----- Problem setup -----
# Given quadratic:  $x^2 + 3x - 1 = 0$ 
a, b, c = 1.0, 3.0, -1.0

# Conic parameters (V, u, f)
V, u, f = quadratic_to_conic(a, b, c)
```

Python Code

```
# x-axis as the line:  $y = 0 \rightarrow h = (0,0)$ ,  $m = (1,0)$ 
h = np.array([0.0, 0.0], dtype=float)
m = np.array([1.0, 0.0], dtype=float)

# ----- Solve via lineconic intersection -----
status, kappa, X = line_conic_intersection(V, u, f, h, m)

# Roots are the x-coordinates of intersection points with  $y=0$ 
roots = []
if status >= 1:
    roots = sorted([float(X[0, 0]), float(X[1, 0])]) if status ==
        2 else [float(X[0, 0])]

print("Status (2:two real, 1:tangent, 0:none):", status)
print("kappa values:", kappa)
print("Intersection points (x,y):\n", X)
print("Roots (x-intercepts):", roots)
```

```
# ----- Plot -----  
xs = np.linspace(-5, 2, 600)  
ys = a*xs**2 + b*xs + c  
  
plt.figure()  
plt.plot(xs, ys, label=rf"$y={a:.0f}x^2{b:+.0f}x{c:+.0f}$")  
plt.axhline(0, linestyle="--", linewidth=1, label="x-axis")
```

```
if status >= 1:
    plt.scatter([X[0,0]], [0.0], s=60, zorder=3, label=f"root {X
        [0,0]:.3f}")
if status == 2:
    plt.scatter([X[1,0]], [0.0], s=60, zorder=3, label=f"root
        {X[1,0]:.3f}")

plt.xlabel("x"); plt.ylabel("y")
plt.title("Roots via lineconic intersection (vectors & matrices)"
    )
plt.grid(True)
plt.legend()
plt.savefig("fig1.png")
plt.show()
```